

bg.py – Behavioral Graph Pipeline Architecture

How the Profile IQ analysis pipeline works from Snowflake to CSV.

1. Entry points

- Terminal: `main()` → `connect_snowflake()`; `perform_full_universe_scan(conn, brands, s1, e1)`; `run_full_pipeline(conn, ...)`.
- Dashboard: `app.run_analysis()` imports `bg`, calls `bg.connect_snowflake()`, then `bg.run_full_pipeline(conn, project_name, brands, sample_start, sample_end)`.

2. Snowflake connection & warehouse

- `connect_snowflake()`: Uses `SNOWFLAKE_USER`, `SNOWFLAKE_TOKEN` or `SNOWFLAKE_PASSWORD`, `BEHAVIORGRAPH6X` warehouse.
- `run_full_pipeline` starts by: `USE WAREHOUSE BEHAVIORGRAPH6X; ALTER WAREHOUSE ... SET WAREHOUSE_SIZE = '6X-Large';` SE

3. Universe scan (optional, called before `run_full_pipeline`)

- `perform_full_universe_scan(conn, brands, start_date, end_date, purchasers_only)`: Counts distinct `UIDs` in `PROCESSEDCLICKSTREAM.PUBLIC`.
- Result stored on `run_full_pipeline.universe_size`; used for sample size inflation (65x down to 1x, cap 10M) and for dynamic sampling (e.g. 10

4. UID cohort & clickstream sampling (inside `run_full_pipeline`)

- Two paths: (A) Fast path: streaming aggregation → `MAPPED_EVENTS` → `TEMP_UIDS`. (B) Default: `brand_filter` + optional `purchasers_only`.
- Date range and universe size drive `sample_rate` (e.g. 10%–50%) and `max_uids` (e.g. 10K–600K). Demographics filter can join `USER_DATA`.
- Eligible `UIDs`: `ELIGIBLE_UIDS = TEMP_SAMPLED_UIDS ■ CLICKSTREAM_FINAL (sample_start..sample_end, brand_filter), HAVING CO`

5. Behavioral data from Snowflake (CTEs → `pandas`)

- `PRE_SAMPLED_CLICKSTREAM`: `CLICKSTREAM_FINAL SAMPLE(clickstream_sample_pct) ■ ELIGIBLE_UIDS, DELIVERED` in [behavior
- `BEHAVIOR_EVENTS`: `PRE_SAMPLED_CLICKSTREAM ■ BEHAVIORALGRAPH.PUBLIC.HOST_MAPPING (COMMON_NAME → Brand,`
- `BEHAVIOR_WITH_DEMOS / BEHAVIORAL_SPLIT`: Joins and splits `HostSection`, `InterestRaw`, `MPC_TRIM` into category columns.
- Final Snowflake result: large CTE query that aggregates by `Column (category)`, `Value (brand/interest)`, with demographics from `USER_DATA`

6. DataFrame normalization & ESPN Layer 1

- Columns normalized: `normalize_category_name(Column)`, `normalize_demo_value(Value)`.
- `consolidate_espn_brands(df)`: Merge ESPN+ into ESPN; set all ESPN entries to the single highest value across categories; remove duplicates.
- Demographics and behavioral data split: `df_demo` vs `df_behavior` (everything not in demo fields or Sample Size).

7. Sample size (intelligent inflation)

- `get_final_sample_size(conn, is_genpop, ...)`: Uses `run_full_pipeline.universe_size` (from universe scan). Bounded to 10M cap.
- Inflation: try 65x, then 55x, 25x, 5x, 2.5x, 1x so `inflated_sample_size ≤ 10M`. `final_sample_size = min(inflated, 10M)`. GenPop: fixed 10M.
- Raw numbers later derived from $(\text{percentage}/100) \times \text{final_sample_size}$.

8. Behavioral processing (noise, caps, merge)

- Behavior percentages: `safe_float_convert`, ± 0.5 jitter, `add_dirichlet_noise(alpha=0.05)`, `add_gaussian_noise_to_behavior` (per category renor
- `apply_category_caps`: `ensure_google_fiber_not_top_7`, `cap_specific_brands` (BYD, Rivian, Centene, etc.), category-specific rules.
- Demographics: `enforce_demographic_minimums`; optional `ensure_demographic_consistency` with previous run. Merge `df_demo` + `df_behavior`

9. Previous run & brand input 100%

- If `previous_file_path`: `load_previous_run_data()` → `previous_demo_lookup`, `previous_behavioral_lookup`. `add_previous_run_column()` adds c

- `enforce_input_brand_100(df_final, brands)`: Set input brand rows to 100% in every category where they appear.

10. Boosts (order matters)

- `boost_all_behavioral_by_2x`: DISABLED (organic only).
- `boost_sports_categories_by_436x`: NFL/NBA/WNBA/MLB 40x then $\div 4$ later; NHL 4.36x then $\div 4$; other sports 4.36x. Cross-category: e.g. LA Lakers to other sports.
- `boost_search_engine_ai_custom`: Google 66x, top 4 @ 33x, others 5–11x. `boost_streaming_platform_custom`: Netflix 15x, Hulu 12x. `divide_by_2`.
- `enforce_cross_category_brand_consistency`: For each brand, set all category rows to the max percentage (and max raw) across categories.

11. Final divisions & ESPN Layers 2–3

- `divide_sports_categories_by_4`: NFL/NBA/WNBA/NHL/MLB. `enforce_sports_global_brand_consistency`.
- `enforce_espn_consistency_final`: ESPN exact match only; set all ESPN % and raw to max across categories; update US Gen Pop.
- `divide_espn_by_2_final`: All ESPN values $\div 2$. `enforce_input_brand_100` again (final 100% check). `enforce_parental_status_sum_to_100`.

12. Output

- Column order: CATEGORY_ORDER (INPUT_METADATA, BRAND INPUT, SAMPLE SIZE, demographics, behavioral categories...). Sort by CATEGORY_ORDER.
- Rename Percentage $\rightarrow$ Category Share. Drop internal columns. Remove dash variants (`remove_dash_variants_from_output`).
- `df_final.to_csv(final_file, index=False)`. `final_file = ~/Desktop/Behavioral_Graph/{project_name}_{timestamp}.csv` (or app overrides path when set).

13. Key data sources (Snowflake)

- `PROCESSEDCLICKSTREAM.PUBLIC.CLICKSTREAM_FINAL`: UID, COMMON_NAME, DELIVERED, URL.
- `BEHAVIORALGRAPH.PUBLIC.HOST_MAPPING`: Brand, Category, Section, Most Purchased Categories (maps COMMON_NAME to behavior).
- `PROCESSEDUSERFILES.PUBLIC.USER_DATA_SANITIZED`: demographics (optional join for filtered UID sampling).

14. S3 (optional)

- `check_s3_for_existing_results(brand, start_date, end_date)`: Look up existing CSV in dashboard-inputs. `upload_result_to_s3(file_path, brand)`.